```
#include <ServoTimer2.h>

// Motor A connections
int enA = 3;
int in1 = 4;
int in2 = 5;
// Motor B connections
int enB = 8;
int in3 = 6;
int in4 = 7;

//payload servo
int servoPayloadPin = 13;
int dropAngle = 500;
int holdAngle=2000;
ServoTimer2 payloadServo;

//ultrasonic sweep servo
int servoUltrasonicPin = 9;
int ultrasonicAngle = 1500;
ServoTimer2 ultrasonicServo;

//ultrasonic sensor
int distance_in = 100;
int trigPin = A1;
int echoPin = A0;

#define msperFoot 555 //time to drive a foot (ms)
#define totalDist (msperFoot*30) //total travel distance (ms)
#define usperFoot 1818 //round trip microseconds/ft for sonar
#define sideDist 5 //distance to travel sideways to get around obstacle
#define dodgeDist 6 //distance to drive to get past obstacle
#define triggerDist 3 //distance to obstacle when dodge routine is
triggered
#define maxDist 6 //max distance for valid sensor reading
#define minDist 1 //min dist for valid sensor reading

int duration_us = 0; //raw microseconds of sensor

float filterConst = 0.05;  //alpha constant for the exponential filter
```

```
float sensorVal = maxDist*usperFoot; //the variable for the filtered
sensor distance (microseconds)

unsigned long startTime = 0;  //start of the mission

unsigned long lastRead = 0;  //variable for storing the time of the last
read of the sensor (for rate control)

int readInterval=20000;  //target interval for reading the sensor (us)

void setup() {
  // Set all the motor control pins to outputs
  pinMode(enA, OUTPUT);
  pinMode(enB, OUTPUT);
  pinMode(in1, OUTPUT);
  pinMode(in2, OUTPUT);
  pinMode(in3, OUTPUT);
  pinMode(in4, OUTPUT);

  // Turn off motors - Initial state
  digitalWrite(in1, LOW);
  digitalWrite(in2, LOW);
  digitalWrite(in3, LOW);
  digitalWrite(in4, LOW);

  //Ultrasonic Rig setup
  pinMode(trigPin, OUTPUT);
  pinMode(echoPin, INPUT);

  payloadServo.attach(servoPayloadPin);
  payloadServo.write(holdAngle);

  ultrasonicServo.attach(servoUltrasonicPin);
  ultrasonicServo.write(ultrasonicAngle);

  delay(10000); //delay to allow rover to be set down
  drive(); //start driving
  startTime= millis(); //set the start timer
}
```

```
void loop() {
  if(micros() - lastRead > readInterval){
    lastRead=micros();
    digitalWrite(trigPin, HIGH);
    delayMicroseconds(30);
    digitalWrite(trigPin, LOW);
    duration_us = pulseIn(echoPin, HIGH);
    if(duration_us>(minDist*usperFoot) &&
duration_us<(maxDist*usperFoot)){ //hard filter rejecting any readings not
in the range mindist to maxdist
      sensorVal=sensorVal+(filterConst*(duration_us-sensorVal));  //raw
value feeds into sensorval through exponential filter
    }
  }
  if (sensorVal<triggerDist*usperFoot){  //excecute dodge sequence
    stop();
    unsigned long distTraveled = millis()-startTime;
    turnLeft();
    drive();
    delay(msperFoot*sideDist);
    stop();
    turnRight();
    drive();
    delay(msperFoot*dodgeDist);
    stop();
    turnRight();
    drive();
    delay(msperFoot*sideDist);
    stop();
    turnLeft();
    drive();
    delay(totalDist-distTraveled-(msperFoot*dodgeDist));
    stop();
    payloadServo.write(dropAngle);
    delay(99999);
  }
}

void drive() {
```

```
  // Turn on motors
  digitalWrite(in1, LOW);
  digitalWrite(in2, HIGH);
  digitalWrite(in3, HIGH);
  digitalWrite(in4, LOW);

  analogWrite(enA, 255);
  analogWrite(enB, 255);

  delay(20);

  analogWrite(enA, 210);
  analogWrite(enB, 200);
}

void stop() {
  digitalWrite(in1, LOW);
  digitalWrite(in2, LOW);
  digitalWrite(in3, LOW);
  digitalWrite(in4, LOW);
  delay(500);
}

//Turn Left 90 degrees
void turnLeft() {
  // Turn on motors
  digitalWrite(in1, LOW);
  digitalWrite(in2, LOW);
  digitalWrite(in3, LOW);
  digitalWrite(in4, HIGH);

  analogWrite(enA, 255);
  analogWrite(enB, 255);

  delay(20);

  // Speed analog values for motors, change for testing
  int LeftSpeed = 0;
  int RightSpeed = 255;
  analogWrite(enA, LeftSpeed);
```

```
  analogWrite(enB, RightSpeed);

  delay(490);

  // Now turn off motors
  digitalWrite(in1, LOW);
  digitalWrite(in2, LOW);
  digitalWrite(in3, LOW);
  digitalWrite(in4, LOW);

  delay(500);

}

//Turn Right 90 degrees
void turnRight() {
  // Turn on motors
  digitalWrite(in1, HIGH);
  digitalWrite(in2, LOW);
  digitalWrite(in3, LOW);
  digitalWrite(in4, LOW);

  analogWrite(enA, 255);
  analogWrite(enB, 255);

  delay(20);

  // Speed analog values for motors, change for testing
  int LeftSpeed = 255;
  int RightSpeed = 0;
  analogWrite(enA, LeftSpeed);
  analogWrite(enB, RightSpeed);

  delay(475);

  // Now turn off motors
  digitalWrite(in1, LOW);
  digitalWrite(in2, LOW);
  digitalWrite(in3, LOW);
  digitalWrite(in4, LOW);
```

```
  delay(500);
}
```